

B.Sc. Information Technology — Semester VI

AI-Driven Software Development (AISD)

Course Code: 25UBIT607

UNITS 2, 3 & 4

Complete Exam-Ready Notes

-  Unit 2 — AI-Powered Code Generation & Refactoring
-  Unit 3 — AI for Testing, Documentation & APIs
-  Unit 4 — Software Quality Metrics & AI Analysis

Credits: 4 | Max Marks: 150

UNIT 2 — AI-Powered Code Generation & Refactoring

1. AI-Powered Code Generation

1.1 What is AI Code Generation?

AI code generation is the process where AI tools automatically write code based on natural language descriptions, comments, or existing code context. It reduces manual coding effort and speeds up development.

1.2 How AI Generates Code

Step	Process
1. Training	LLMs trained on large collections of programming code from repositories, documentation, and tutorials
2. Tokenization	Code and text are broken into smaller units called tokens
3. Transformer Architecture	Uses attention mechanisms to understand relationships between tokens
4. Context Understanding	Model analyzes the prompt and surrounding code context
5. Probability Prediction	LLM predicts the most probable next token based on previous tokens
6. Sequential Generation	Tokens generated step by step until full code solution is produced

 Quality of code depends on the clarity and structure of the prompt provided by the user

1.3 Types of Code Generation

Type	Description	Example
CRUD Generation	AI generates Create, Read, Update, Delete operations from database schema	Generate CRUD API for Student table in MongoDB

Type	Description	Example
Sorting/Searching Algorithms	AI generates classic algorithms from description or comment	Comment: 'Sort array using Quick Sort' → AI generates full function
UI Component Generation	AI generates HTML/CSS/React components from description	Comment: 'Create responsive login form' → AI generates complete form
Boilerplate Code	AI generates repetitive setup code for frameworks	Express server setup, React component structure, API skeleton
API Generation	AI generates REST API endpoints from natural language requirements	Describe endpoints in plain English → AI generates routes + handlers

1.4 Writing Code Using GitHub Copilot in VS Code

How Copilot Integrates with VS Code:

- Install GitHub Copilot extension from VS Code Marketplace
- Log in with GitHub account to activate Copilot
- Copilot analyzes current file, comments, and surrounding code
- Provides inline code suggestions while developers type
- Suggestions appear as gray text in the editor
- Press Tab to accept suggestion — other options via shortcuts
- Copilot Chat feature allows asking questions about code directly in editor

Example — Code from Comment:

```
# Calculate factorial of a number
# Return the result

# Copilot generates:
def factorial(n):
    if n == 0: return 1
    result = 1
    for i in range(1, n+1):
        result *= i
    return result
```

1.5 Generating CRUD Applications Using AI

- Developers describe the data model or database table in natural language
- AI generates complete CRUD operations — Create, Read, Update, Delete
- Generates routes, controllers, database queries automatically
- Supports frameworks: Express.js (Node), Flask (Python), Spring Boot (Java)

📌 *Example: Prompt 'Create CRUD API for Student with name, age, marks using MongoDB and Express' → AI generates all routes*

1.6 Generating Sorting & Searching Algorithms

- AI generates classic algorithms from comments or descriptions
- Supports: Bubble Sort, Quick Sort, Merge Sort, Binary Search, Linear Search
- Generates correct implementation with proper time complexity

📌 *Example: Comment '// Binary search on sorted array, return index' → Copilot generates complete function*

1.7 Generating UI Components Using AI

- AI generates HTML, CSS, JavaScript code automatically while typing
- Comment describing component → Copilot generates full UI component
- Supports React, Angular, Vue.js, Bootstrap
- Generates event handlers (onClick, form validation, input handling)
- Generates CSS or Tailwind classes automatically

📌 Example: 'Create a responsive login form with email and password fields' → AI generates complete form

2. AI-Assisted Code Refactoring

2.1 What is Code Refactoring?

Code refactoring is the process of modifying the internal structure of existing code without changing its external functionality. The goal is to make code cleaner, more readable, more efficient, and easier to maintain.

Why Refactoring is Needed:

- Duplicate code — same logic written multiple times
- Poor variable names — unclear what variables represent
- Inefficient logic — slow or complex algorithms
- Large complex functions — one function doing too many things

2.2 Refactoring Legacy Code Using AI

Legacy code refers to old software code that is still in use but may be difficult to maintain or modify.

Problems with Legacy Code	How AI Helps
Poor documentation	AI generates missing comments and documentation automatically
Outdated programming practices	AI suggests modern coding patterns and best practices
Complex logic	AI breaks complex functions into smaller, readable modules
Lack of modular design	AI restructures code into proper modules and functions

2.3 AI Techniques Used for Refactoring

Technique	Description	Example
Static Code Analysis	Examines code WITHOUT executing it to detect code smells, vulnerabilities, performance issues	SonarQube — detects unused variables, security holes
Pattern Recognition	AI identifies common programming patterns and suggests improvements	Recognizes nested loops that can be simplified

Technique	Description	Example
Automated Code Transformation	AI renames variables, breaks large functions, optimizes loops	Splits 200-line function into 5 focused functions

2.4 Benefits of AI Code Generation & Refactoring

Benefit	Description
Faster Development	AI generates code quickly, reducing development time significantly
Reduced Errors	AI suggestions help detect bugs early in development
Improved Code Quality	Refactoring improves readability and maintainability
Better Productivity	Developers focus on complex problem solving instead of repetitive coding
Faster Learning	Students and junior developers learn coding patterns from AI suggestions

2.5 Limitations of AI-Based Code Generation

- Sometimes produces incorrect or incomplete code — needs review
- Generated code may have security vulnerabilities if not checked
- AI does not understand business logic — only patterns
- Poor prompts lead to poor code quality
- Over-reliance on AI may reduce developer problem-solving skills
- Code may not follow team-specific standards or conventions

2.6 GitHub Copilot vs TabNine — Comparison

Feature	GitHub Copilot	TabNine
Developer	GitHub and OpenAI	TabNine company
AI Model	Large language models trained on large code datasets	Deep learning models trained on code patterns
Suggestion Type	Generates full functions, code blocks, and explanations	Mainly focuses on code autocompletion
Context Awareness	Uses larger project context for suggestions	Primarily uses local code context
Cloud Usage	Mostly cloud-based AI processing	Offers both cloud and local models
Best Use Case	Advanced AI coding assistant	Fast code completion and privacy-focused environments
Integration	VS Code, Visual Studio, JetBrains IDEs	Works with many IDEs including VS Code, IntelliJ, PyCharm

UNIT 3 — AI for Testing, Documentation & APIs

3. AI in Software Testing

3.1 Role of AI in Software Testing

AI transforms software testing by automating test case generation, predicting defect-prone areas, and performing regression testing — reducing manual effort and improving test coverage.

AI Testing Capability	Description	Example
Test Case Generation	AI automatically generates test cases for different scenarios	AI generates test for valid login, invalid login, empty fields, SQL injection
Defect Prediction	ML models predict which modules are likely to contain bugs	Module with high complexity flagged as high-risk before testing
Regression Testing	AI re-runs existing tests after code changes to verify nothing broke	After adding 'Forgot Password' — AI re-tests entire login module
UI Testing	AI visually tests web pages across browsers automatically	Selenium + AI tests same page on Chrome, Firefox, Edge
Performance Testing	AI identifies performance bottlenecks under load	AI detects slow database query under 1000 concurrent users

3.2 Test Case Generation Using AI

- AI analyzes source code, requirements, and comments to generate test cases
- Generates both positive tests (valid inputs) and negative tests (invalid inputs)
- Covers edge cases that developers might miss
- Supports unit testing, integration testing, and end-to-end testing

Example — AI generates test cases for Login function:

```
Test 1: Valid username + valid password → Should login successfully
Test 2: Valid username + wrong password → Should show error
Test 3: Empty username → Should show validation error
Test 4: SQL injection in username → Should be blocked
Test 5: Very long username (1000 chars) → Should handle gracefully
```

3.3 Unit Testing Using AI

Unit testing verifies individual functions or modules work correctly in isolation.

- AI generates unit test functions from source code automatically
- Identifies input-output pairs for testing
- Generates mock objects and stubs for dependencies
- Frameworks used: JUnit (Java), pytest (Python), Jest (JavaScript)

Example — AI generates unit test:

```
# Function to test
def add(a, b):
    return a + b

# AI generates unit test:
def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
    assert add(0, 0) == 0
```

3.4 AI in Static Code Analysis

Static code analysis is the process of examining source code WITHOUT executing the program to detect errors and vulnerabilities.

AI Static Analysis Feature	Description
Automated Code Inspection	AI analyzes source code to detect syntax errors and coding issues
Security Vulnerability Detection	Identifies SQL injection, XSS (Cross-site scripting), Buffer overflow
Code Quality Improvement	Suggests better coding practices and refactoring opportunities
Pattern Recognition	Identifies problematic coding patterns based on training data
CI/CD Integration	AI tools integrate with CI/CD pipelines to analyze code automatically
Reducing False Positives	ML models improve accuracy in detecting real issues vs false alarms

4. AI in Documentation

4.1 What is AI-Assisted Documentation?

AI-assisted documentation is the automatic generation of code comments, function descriptions, README files, and API documentation using AI tools. This saves developers significant time and ensures documentation is always up to date.

4.2 Types of Documentation AI Can Generate

Documentation Type	Description	Example
Inline Comments	AI adds comments explaining what each line of code does	// Check if user exists in database before login
Function/Method Documentation	AI generates docstrings explaining purpose, parameters, return values	"""Calculates student grade. Args: marks (int). Returns: grade (str)"""
API Documentation	AI generates Swagger/OpenAPI documentation for REST endpoints	POST /login — Parameters: username, password — Returns: JWT token

Documentation Type	Description	Example
README Files	AI generates project overview, setup instructions, usage examples	Full README with installation steps, API endpoints, examples
User Manuals	AI converts technical documentation into user-friendly guides	Step-by-step user guide from code comments

4.3 Documentation Assistance by Copilot

- Write a function → Copilot automatically suggests documentation above it
- Generates JSDoc (JavaScript), PyDoc (Python), Javadoc (Java) format comments
- Explains complex algorithms in plain English comments
- Updates documentation when code changes

📌 Example: Type function signature → Copilot auto-generates complete docstring with param descriptions

5. AI in API Generation

5.1 Role of AI in Automatically Generating APIs

AI can assist developers in automatically generating complete REST APIs based on natural language requirements or database structures — drastically reducing development time.

API Generation Type	Description	Example
Requirement-Based	Describe functionality in natural language → AI generates API endpoints	Prompt: 'Create API for student management' → AI generates all routes
Database-Based	AI generates APIs automatically from database schemas	Upload ER diagram → AI generates CRUD endpoints for each table
Framework-Specific	AI generates framework-specific API code	Flask, Django, Express.js, Spring Boot templates
Automatic Documentation	AI generates Swagger/OpenAPI docs alongside the API	Complete API documentation with request/response examples
Security Implementation	AI adds authentication to generated APIs	Adds JWT tokens, API keys, OAuth automatically
Test Case Generation	AI generates test cases for API endpoints	Postman test scripts for each generated endpoint

5.2 Role of AI Tools in SDLC Stages

SDLC Stage	AI Role	Tools/Example
Requirement Analysis	NLP analyzes requirements, converts to user stories, detects inconsistencies	Converts 'User should track orders' → functional requirements
System Design	Suggests architectures, design patterns, generates UML diagrams	AI generates class diagram from requirements description
Coding / Development	Code suggestions, auto-completion, code snippets	GitHub Copilot, TabNine, CodeWhisperer
Testing	Generates test cases, predicts defects, regression testing	AI generates unit tests, identifies high-risk modules
Debugging	Analyzes logs, identifies error causes, suggests bug fixes	AI explains error trace and suggests fix
Deployment	Configures CI/CD pipelines, predicts deployment risks	AI warns about performance issues before release
Maintenance	Predictive maintenance, identifies performance bottlenecks	AI alerts when response time degrades

UNIT 4 — Software Quality Metrics & AI Analysis

6. Software Quality Metrics

6.1 What are Software Quality Metrics?

Software quality metrics are measurable values used to evaluate the quality of software. They help teams predict defects, improve maintainability, and ensure reliable software systems.

6.2 Coupling

Coupling refers to the degree of dependency between modules in a software system.

Type	Description	Impact on Maintainability
High Coupling	Modules depend heavily on each other	Changes in one module affect many others — difficult to maintain
Low Coupling	Modules are independent of each other	Easy to modify or replace components — improves maintainability

📌 Goal: Always aim for LOW COUPLING — modules should be as independent as possible

6.3 Cohesion

Cohesion refers to the degree to which elements within a module belong together and perform a single focused task.

Type	Description	Impact on Maintainability
High Cohesion	Module performs a single well-defined task	Improves readability, testability, and maintainability
Low Cohesion	Module performs many unrelated tasks	Difficult to understand, test, and maintain

📌 Goal: Always aim for HIGH COHESION — each module should do ONE thing well

✅ **Golden Rule: Low Coupling + High Cohesion = Best maintainable software design**

6.4 Cyclomatic Complexity

Cyclomatic complexity is a software metric used to measure the complexity of a program's control flow. It counts the number of independent paths through the code.

Formula:

$$M = E - N + 2P$$

Where:

E = Number of edges in the control flow graph

N = Number of nodes

P = Number of connected components (usually 1)

Cyclomatic Complexity	Meaning	Risk Level
1 - 10	Simple, well-structured code — few decision points	Low risk — easy to test and maintain
11 - 20	Moderate complexity — some branching logic	Medium risk — needs careful testing
21 - 50	Complex code — many decision paths	High risk — difficult to test fully
50+	Extremely complex — almost impossible to test completely	Very High risk — must refactor

Impact on Software Reliability:

- High Cyclomatic Complexity → complex code → difficult to test → higher chance of bugs
- Low Cyclomatic Complexity → simpler code → fewer test cases needed → more reliable
- Cyclomatic complexity indicates minimum number of test cases needed for full path coverage

6.5 Importance of Software Quality Metrics in Defect Prediction

Benefit	Description
Early Detection of Errors	Metrics identify modules likely to contain defects before testing begins
Risk Identification	High complexity or large modules have higher defect probability
Better Testing Strategy	Testing efforts focused on high-risk components first
Improved Software Reliability	Monitoring metrics helps maintain stable and reliable systems
Efficient Resource Allocation	Developers and testers can prioritize problematic modules

7. AI in Code Quality & Analysis

7.1 Role of AI in Static Code Analysis

AI-powered static code analysis tools examine source code WITHOUT executing it to automatically detect bugs, security issues, and code quality problems.

AI Analysis Feature	Description
Automated Inspection	AI scans entire codebase for syntax errors, coding issues, and anti-patterns
Security Detection	Identifies SQL injection, XSS, buffer overflow, insecure API usage
Code Smell Detection	Finds duplicate code, long methods, complex conditionals
Pattern Recognition	Learns from millions of code samples to identify problematic patterns
CI/CD Integration	Runs automatically on every code commit in the pipeline
Reduced False Positives	ML improves accuracy by learning which alerts are real issues

7.2 Prompt Engineering — Definition & Importance

Prompt engineering is the process of designing and refining input instructions given to an AI system to produce accurate, relevant, and useful outputs.

Aspect	Advantages	Disadvantages
Quality	Improves quality and accuracy of AI outputs	Requires experience to write effective prompts
Output	Helps generate structured and relevant code	Poor prompts can produce incorrect outputs
Time	Saves development time significantly	AI may still generate misleading responses
Customization	Allows customization of AI responses	Dependent heavily on prompt quality

7.3 Code Refactoring — Advantages & Disadvantages

✔ Advantages	✖ Disadvantages
Improves code readability	Requires time and effort to refactor
Makes code easier to maintain	Risk of introducing new bugs if not tested properly
Reduces technical debt	May need extensive regression testing after refactoring
Improves performance	Team members may resist changes to familiar code
Easier to extend with new features	Short-term slowdown in development speed

7.4 Code Generation — Advantages & Disadvantages

✔ Advantages	✖ Disadvantages
Speeds up development significantly	Generated code may contain bugs or security issues
Reduces repetitive manual coding	Code may not follow team-specific conventions
Helps beginners learn from examples	Over-reliance reduces developer skill development
Generates boilerplate instantly	AI lacks true understanding of business context
Consistent code structure	Still requires human review before deployment

7.5 Benefits & Drawbacks of AI in SDLC

SDLC Area	✔ Benefit	✖ Drawback
Coding	Faster development with auto-suggestions	Generated code needs review and testing

SDLC Area	✅ Benefit	❌ Drawback
Testing	Automatic test case generation saves time	AI cannot fully replace human judgment in testing
Deployment	Predicts risks, configures CI/CD pipelines	AI recommendations may not always be accurate
Documentation	Auto-generates comments and docs	May generate generic or inaccurate documentation
Overall	Improves productivity and reduces errors	Risk of over-dependence on AI tools

★ QUICK REVISION SUMMARY — UNITS 2, 3 & 4

Topic	Key Points
AI Code Generation	LLMs generate code from prompts Tokenization → Transformer → Probability prediction → Sequential output
CRUD Generation	AI generates full Create/Read/Update/Delete from description Supports Express, Flask, Spring Boot
UI Component Generation	Comment describing component → AI generates HTML/CSS/React Supports React, Angular, Vue, Bootstrap
Boilerplate Code	AI generates repetitive setup code — server setup, API skeleton, component structure
Code Refactoring	Improves internal structure WITHOUT changing functionality Cleaner, readable, maintainable
Legacy Code	Old difficult-to-maintain code AI modernizes via static analysis, pattern recognition, transformation
AI Refactoring Techniques	Static Code Analysis + Pattern Recognition + Automated Code Transformation
Copilot in VS Code	Install extension → Log in → Tab to accept suggestion → Gray text suggestions Copilot Chat for queries
Copilot vs TabNine	Copilot: full functions, cloud-based TabNine: fast autocompletion, local model option
Test Case Generation	AI generates positive + negative + edge case tests Unit/integration/E2E JUnit, pytest, Jest
Static Code Analysis	Examines code WITHOUT execution Finds security vulns (SQL injection, XSS), code smells
Documentation	Auto-generates inline comments, docstrings, Swagger API docs, README files
API Generation	Natural language → AI generates REST endpoints + documentation + security + test cases
AI in SDLC	Requirement(NLP) → Design(diagrams) → Coding(Copilot) → Testing(auto-cases) → Deploy(CI/CD) → Maintain
Coupling	High = modules depend heavily (bad) Low = independent modules (good) Aim for LOW coupling
Cohesion	High = one task per module (good) Low = multiple tasks (bad) Aim for HIGH cohesion
Cyclomatic Complexity	$M = E - N + 2P$ High = complex, many bugs Low = simple, reliable Sets min test cases needed
Defect Prediction	Quality metrics identify high-risk modules Focus testing on complex/large modules
Prompt Engineering	Design clear instructions for AI Advantages: quality output, saves time Need: experience to write well

Topic	Key Points
AI Limitations	May produce incorrect code No business logic understanding Needs human review always

Important Exam Questions

Unit 2:

- Explain AI-powered code generation — how AI generates code step by step (5 marks)
- Explain AI-assisted code refactoring — what it is, why needed, AI techniques used (7 marks)
- What is legacy code? How does AI help modernize legacy code? (5 marks)
- Explain benefits and limitations of AI-based code generation (5 marks)
- Compare GitHub Copilot and TabNine (5 marks)
- How does Copilot integrate with VS Code? Explain with steps (5 marks)

Unit 3:

- Explain the role of AI in software testing — test case generation, defect prediction (5 marks)
- Explain AI in static code analysis — what it does and how it works (5 marks)
- Explain AI-assisted documentation — types and how AI generates it (5 marks)
- Explain role of AI in automatically generating APIs (5 marks)
- Explain role of AI tools in different stages of SDLC (7 marks)

Unit 4:

- Explain Coupling and Cohesion with their types and impact on maintainability (7 marks)
- What is Cyclomatic Complexity? Explain formula and impact on reliability (5 marks)
- Explain importance of software quality metrics in defect prediction (5 marks)
- What is Prompt Engineering? Explain advantages and disadvantages (5 marks)
- Explain benefits and drawbacks of AI in SDLC (5 marks)

✅ All the best! AISD Units 2, 3 & 4 — Fully Covered! 🚀